A

PROJECT REPORT

ON

" F.B.C.R(FM BASED COLLEGE RADIO)"

Submitted in partial fulfillment of the requirements for the award of

DIPLOMA

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

SUBMITTED BY

V. SRI SAI PURNA JANAKA DATTA [23241-EC-094]

Under the esteemed guidance of

Mrs. B. RADHIKA
M. Tech
Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
(Affiliated to SBTET, 2nd Shift Polytechnic College)
Medibowli, Meerpet, Hyderabad -500097

2025-2026

TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
(Affiliated to SBTET, 2nd Shift Polytechnic College)
Medibowli, Meerpet, Hyderabad -500097

CERTIFICATE

This is to certify that the project work entitled

" F.B.C.R(FM BASED COLLEGE RADIO)"

SUBMITTED BY

V. SRI SAI PURNA JANAKA DATTA [23241-EC-094]

Submitted in Partial fulfillment of the requirement for the award of Diploma in the department of ELECTRONICS AND COMMUNICATION ENGINEERING, TKRCET, SBTET, TELANGANA, Hyderabad is a record of Bonafide work carried out by him during the year 2025-26 under our guidance and supervision.

INTERNAL GUIDE **CO-ORDINATOR**

Mrs. B. RADHIKA **Mrs. K. UMA RANI**

H. O. D. **DIPLOMA I/C** **PRINCIPAL**

Mrs. K. UMA RANI

EXTERNAL EXAMINER

TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
(Affiliated to SBTET, 2nd Shift Polytechnic College)
Medibowli, Meerpet, Hyderabad -500097

DECLARATION

I declare that the Project entitled "F.B.C.R (FM BASED COLLEGE RADIO)" is original and Bonafide work of me in partial fulfillment of the requirement for the award of DIPLOMA in DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING, TKRC, SBTET, Telangana, Hyderabad, under the able guidance of Mrs. B. RADHIKA, Assistant Professor, DECE Department and has not been copied from any earlier report.

SUBMITTED BY

V. SRI SAI PURNA JANAKA DATTA [23241-EC-094]

TKR COLLEGE OF ENGINEERING AND TECHNOLOGY
(Affiliated to SBTET, 2nd Shift Polytechnic College)
Medibowli, Meerpet, Hyderabad -500097

ACKNOWLEDGEMENT

Behind every achievement lies an unfathomable sea of gratitude to those who activated it, without them it would have been a dream. I lay the words of gratitude to them who guided me to the ocean of knowledge.

First of all, I would like to thank our respected Principal Sir, for giving me an opportunity to excel our skills in the project.

I would like to extend our gratitude for successful completion of this project under the supervision of our beloved Head of the department Mrs. K. UMA RANI, Assistant Professor, Department of DECE, and GUIDE Mrs. RADHIKA, Assistant professor, Department of DECE, TKRCET.

I wish My Profound thanks and gratitude to our esteemed Co-Ordinator Mrs. K. UMA RANI, Assistant Professor, Department of DECE, TKRCET, for the coordination and assistance in completing of My Project Successfully. I have enjoyed during the entire course of the project work.

I would like to thank everyone who has been involved in the progress of this Project, whose contribution have added a lot of Value.

SUBMITTED BY

V. SRI SAI PURNA JANAKA DATTA [23241-EC-094]

ABSTRACT

This project aims to design and implement an FM radio system using a Raspberry Pi that can transmit Radio signals within FM frequency range. Utilizing software-defined radio (SDR) techniques or GPIO-based transmission, the Raspberry Pi can act as a low-power FM transmitter for Educational and Experimental purposes. The Project demonstrates how modern embedded platforms can be used to replicate traditional analog systems with digital flexibility and minimal hardware.

INDEX

LIST OF FIGURES

LIST OF TABLES

CHAPTER-1
INTRODUCTION

1.0 Overview

FM radio is one of the oldest and most reliable broadcasting technologies used globally for audio transmission. With the advent of microcomputers like Raspberry Pi, it's now possible to digitally stimulate or interface with radio frequency systems. The Raspberry Pi is used as an FM transmitter (using its GPIO pins and software PiFM). This project showcases the capability of the Raspberry Pi to handle real time signal processing and interact with radio frequency hardware through software-defined platforms.

1.1 Existing System

Traditional FM radio systems are built using analog circuits involving RF amplifiers, oscillators, and antennas. These systems lack flexibility in changing frequencies or modulation.

1.2 Proposed System

Due to the limitations and disadvantages of Analog System and Existing System a new system using the Raspberry Pi is being proposed. This is accomplished by using making an FM Transmitter using the Raspberry Pi within the range of 100 meters sweep which is relatively adequate to cover the whole campus (or) the whole area of Industry, Hospital, workplace, etc; The proposed system replaces complex analog circuitry with a Raspberry Pi-based digital system.

Its uses:

* The Raspberry Pi's GPIO pin for FM signal generation using software like PiFM or FM Transmitter.
* Audio input through the Pi’s USB microphone or pre-loaded files.

* Audio output via speakers connected to mobile in receiving part like 3.5mm jack or Bluetooth.

* A Python-based or Linux terminal user interface to select frequency and content.

This system is inexpensive, flexible, and educational, serving as a modern alternative for FM transmission and reception.

CHAPTER-2

COMPONENTS

2.0 Overview

The components of your FM Based College Radio (FBCR) project are essential because each one performs a unique, non-negotiable function that layers up to create a working broadcast system.

2.1 Raspberry Pi

A Raspberry Pi is a series of small, inexpensive, single-board computers (SBCs) developed in the UK by the Raspberry Pi Foundation to promote the teaching of basic computer science.

Specifications

1.Processor: Broadcom BCM2837B0. It's a 64-bit quad-core SoC.

2.clock Speed :1.4 GHz.

3.Core Architecture: quad-core 64-bit ARM Cortex-A53 processor.

4.Size: 85.6 x 56.5 x17(all measurements in mm).

5.Operating System: Raspberry Pi OS/Raspberry pi imager.

6.Power: Micro USB - Voltage :5.1V. Current: 2.5A.

7.GPIO's :40 Pins.

8.Memory :1GB of SDRAM.

9.WiFi: Dual Band 2.4 GHz and 5 GHZ wireless LAN (802.11 b/g/n/ac).

10.Bluetooth :4.2/BLE.

11.Ports and Interfaces:

•HDMI Port: HDMI, Composite (TRRS).

•Camera Connector:15-pin MIPI CSI Port.

•Ethernet: Gigabit Ethernet over USB 2.0 port with maximum throughput of 300 Mbit/s LAN Port.

12.External memory: MicroSD card slot for the operating system and storage purpose.

13.Video output :1080p60

14. Audio Output :3.5mm, 4-pole jack.

Important Full forms:

1.SOC: system on Chip.
2.BCM: Broadcom.
3.ARM: Advanced RISC Machines.
4.USB: Universal Serial Bus.
5.GPIO's: General purpose I/O pins.
6.LPDDR2: Low Power Double Data Rate II Synchronous Dynamic Random-Access Memory.
7.SDRAM: Synchronous Dynamic Random Access Memory.
8.LAN: Local Area Network.
9.BLE: Bluetooth Low energy.
10.HDMI : High-Definition Multimedia Interface.
11.MIPI: Mobile Industry Processor Interface.
12. CSI : Camera serial interface.
13.SD card: Secure Digital Card.

2.2 Antenna

Antenna is used to transmit the radio frequency (RF) signals in the range of FM Signals.

In radio engineering, an antenna is the interface between radio waves Propagating through space and electric currents moving in metal conductors, used with a transmitter or receiver.

In transmission, a radio transmitter supplies an electric current to the antenna's terminals, and the antenna radiates the energy from the current as electromagnetic waves (radio waves). The first antennas were built in 1888 by German physicist Heinrich Hertz in his pioneering experiments to prove the existence of waves predicted by the electromagnetic theory of James Clerk Maxwell. Hertz placed dipole antennas at the focal point of parabolic reflectors for both transmitting and receiving.

Starting in 1895, Guglielmo Marconi began development of antennas practical for long-distance, wireless telegraphy, for which he received a Nobel Prize.

Specifications

1.High Conductivity
2.Corrosion Resistance
3.Ductility and Strength
4.Cost-Effectiveness
5.Versatility
6.Weight
7.Flexibility

2.3 Microphone

Microphone is a Transducer that converts sound waves into an Electrical Signal. The Input we give i.e., the Audio input/ Audio files are given with the help of Mic.

Features of Microphone

1. Dynamic: Active all the time.
2. Condenser: These mics use a charged diaphragm and a backplate to create a capacitor. They are highly sensitive, offer excellent detail and frequency response, and require a power source (like phantom power).

3.Polar Pattern: It's known as the sensitivity of the mic to the sounds from all the directions.
4.Omnidirectional:
Transmits signals in all directions equally.
5.Frequency Response: This is the range of frequencies that a microphone can capture and how it responds to different frequencies. A flat frequency response means the mic captures all frequencies equally. It's frequency response is in audio range (20-20,000 Hz) and it's linear.
6.Impedance: Less Impedance, greater performance.
7. Cardioid: Shaped like a heart, this pattern is most sensitive to sound from the front and rejects sound from the back. It is excellent for isolating a single sound source.

2.4 USB wire

USB i.e., universal serial bus is used to connect the raspberry pi to the microphone. A raspberry pi 3 model B+ consists of four USB ports.

Importance of USB

1.support reliable communication.
2.Ensures clean power supply.
3.Easy to plug/unplug during Development.

2.5 Phone or Desktop

This is used for selecting the FM signal for transmission by knowing the IP (Internet protocol) address of the Raspberry Pi. If the phone is not Samsung, we have to use additional app Called "Network Analyzer".

2.6 Power System

Our system requires a single power supply to raspberry pi connected through the Micro USB socket of the Raspberry Pi.

Power requirements of raspberry pi :
Voltage rating: 5.1V.
Current rating: 2.5A.

A well-organized power supply prevents voltage sags and noise, particularly in mixed digital-analog environments. This segmented power system ensures stability, prevents voltage drops, and minimizes interference among the components.

CHAPTER-3

HARDWARE DESCRIPTION

3.0 Overview

This chapter deals with the basic description about the hardware used in our Project F. B. C. R. This also includes detailed information about choosing the particular components.

3.1 Raspberry Pi

It's used as the RF signal transmitter in the FM range of frequency. In our project F. B. C. R, we using the Raspberry Pi 3 model B+ version.

3.1.0 Advantages/Merits of Raspberry Pi 3 model B+

1. Improved Performance

The 1.4GHz quad-core ARM Cortex-A53 processor (Broad-com BCM2837B0) provides a significant speed boost compared to earlier models.

2. Enhanced Connectivity

This is the major upgrade, which includes:

•Dual-band WIFI: It has 2.4GHz and 5GHz wireless LAN (802.11ac).

•Bluetooth 4.2/BLE: Improved Bluetooth connectivity.

3. Power Over Ethernet (POE) capability

With this new feature an Ethernet cable can be directly connected to raspberry pi. This also eradicates the need of an additional power supply.

4.Versatile I/O

Increase in the number of pins from 26 to 40 pins adds some new functions that can be performed.

5. Operating System

Unlike Micro controllers, the Raspberry Pi 3 Model B+ runs with the Raspberry Pi OS.

6. Multimedia Features

The Features like HDMI, CSI Port etc; Can be used to perform numerous functions.

7. Cost

The cost is low since many abilities can be performed, so it's affordable in price.

3.1.1 Comparison between various versions of Raspberry Pi

The table below gives the speed specifications of various raspberry pi models and generations focusing on the version's release date, form factor and dimensions:

Raspberry Pi Version	Release Date	Form Factor	Dimensions (in mm)
Raspberry Pi 4 Model B	2019-2020	Standard	85.6 x 56.5
Raspberry Pi 3 Model B+	2018	Standard	85.6 x 56.5
Raspberry Pi 3 Model B	2016	Standard	85.6 x 56.5
Raspberry Pi 3 Model A+	2018	Compact	65 x 56.5
Raspberry Pi Zero Wireless with Headers	2017	Mini	65 x 30 x 5
Raspberry Pi Zero Wireless	2016	Mini	65 x 30 x 5
Raspberry Pi Zero	2015	Mini	65 x 30 x 5
Raspberry Pi 2 Model B	2015	Standard	85.6 x 56.5
Raspberry Pi 1 Model B +	2014	Standard	85.6 x 56.5
Raspberry Pi 1 Model B	2012	Standard	85.6 x 56.5
Raspberry Pi 1 Model A+	2014	Compact	65 x 56.5
Raspberry Pi 1 Model A	2013	Standard	85.6 x 56.5

The table below gives the speed specifications of various Raspberry Pi models and generations focusing on the version's weight, General Purpose Input/Output (GPIO), central processing unit (CPU) speed, Cores and Random-access memory (RAM):

Raspberry Pi Version	Weight (in grams)	GPIO	CPU Speed	Cores	RAM
Raspberry Pi 4 Model B	46	40 Pin	1.5 GHz	Quad	1,2,4, or 8 GB
Raspberry Pi 3 Model B+	50	40 Pin	1.4 GHz	Quad	1 GB
Raspberry Pi 3 Model B	40	40 Pin	1.2 GHz	Quad	1 GB
Raspberry Pi 3 Model A+	28	40 Pin	1.4 GHz	Quad	512 MB
Raspberry Pi Zero Wireless with Headers	10	40 Pin	1 GHz	Single	512 MB
Raspberry Pi Zero Wireless	10	40 Pin Unpopulated	1 GHz	Single	512 MB
Raspberry Pi Zero	8	40 Pin Unpopulated	1 GHz	Single	512 MB
Raspberry Pi 2 Model B	42	40 Pin	1.2 GHz	Quad	1 GB
Raspberry Pi 1 Model B+	42	40 Pin	700 MHz	Single	512 MB
Raspberry Pi 1 Model B	38	21 Pin (26 Pin Header)	700 MHz	Single	512 MB
Raspberry Pi 1 Model A+	23	40 Pin	700 MHz	Single	512 MB
Raspberry Pi 1 Model A	30	21 Pin (26 Pin Header)	700 MHz	Single	256 MB

Connectivity Specifications

The table below gives the connectivity specifications of various Raspberry Pi boards focusing on the version's full sized USB ports, other USB and charge methods, power and High-Definition Multimedia Interface (HDMI) ports:

Raspberry Pi Version	Full sized USB Ports	Other USB & Charge Methods	Power	HDMI Ports
Raspberry Pi 4 Model B	2 USB3.0 2 USB2.0	1 USB-C	5.1V at 3A	2 micro-HDMI
Raspberry Pi 3 Model B+	4 USB2.0	1 MIcroUSB	5.1V at 2.5A	HDMI, Composite (TRRS)
Raspberry Pi 3 Model B	4 USB2.0	1 MIcroUSB	5.1V at 2.5A	HDMI, Composite (TRRS)
Raspberry Pi 3 Model A+	1 USB2.0	1 MIcroUSB	5.1V at 3A	HDMI, Composite (TRRS)
Raspberry Pi Zero Wireless with Headers	—	1 MIcroUSB	5.1V at 1.2A	Mini-HDMI,GPIO Composite
Raspberry Pi Zero Wireless	—	1 MIcroUSB	5.1V at 1.2A	Mini-HDMI,GPIO Composite
Raspberry Pi Zero	—	1 MIcroUSB	5.1V at 1.2A	Mini-HDMI,GPIO Composite
Raspberry Pi 2 Model B	4 USB2.0	1 MIcroUSB	5.1V at 1.8A	HDMI, Composite (TRRS)
Raspberry Pi 1 Model B+	4 USB2.0	1 MIcroUSB	5.1V at 1.2A	HDMI, Composite (TRRS)
Raspberry Pi 1 Model B	2 USB2.0	1 MIcroUSB	5.1V at 3A	PAL andNTSC, HDMI or DSI, RCA
Raspberry Pi 1 Model A+	1 USB2.0	1 MicroUSB or GPIO	5.1V at 700mA	HDMI, Composite (TRRS)

Raspberry Pi 1 Model A	1 USB2.0	1	MicroUSB or GPIO	5.1V at 700mA	PAL and NTSC, HDMI or DSI, RCA

The table below gives the connectivity specifications of various Raspberry Pi boards focusing on the version's video out quality, video in, Ethernet, Bluetooth, Wi-Fi and external storage:

Raspberry Pi Version	Video Out Quality	Video In	Ethernet	Bluetooth	Wi-Fi	External Storage
Raspberry Pi 4 Model B	4kp60	CSI Camera Connector	Gigabit Ethernet	Bluetooth 5.0	Dual Band-2.4 GHz and 5GHz	MicroSD
Raspberry Pi 3 Model B+	1080p60	CSI Camera Connector	10/100 Mbit/s	Bluetooth 4.2/BLE	Dual Band-2.4 GHz and 5GHz	MicroSD
Raspberry Pi 3 Model B	1080p60	CSI Camera Connector	10/100 Mbit/s	Bluetooth 4.1	2.4 GHz	MicroSD
Raspberry Pi 3 Model A+	1080p60	CSI Camera Connector	—	Bluetooth 4.2/BLE	Dual Band-2.4 GHz and 5GHz	MicroSD
Raspberry Pi Zero Wireless with Headers	1080p60	CSI Camera Connector	—	Bluetooth 4.1	2.4 GHz	MicroSD
Raspberry Pi Zero Wireless	1080p60	CSI Camera Connector	—	Bluetooth 4.1	2.4 GHz	MicroSD
Raspberry Pi Zero	1080p60	CSI Camera Connector	—	—	—	MicroSD

Raspberry Pi 2 Model B	1080p60	CSI Camera Connector	10/100 Mbit/s	___	___	MicroSD
Raspberry Pi 1 Model B +	1080p60	CSI Camera Connector	10/100 Mbit/s	___	___	MicroSD

3.2 Antenna

3.2.0 Introduction

In our Project FBCR we are using copper wired antenna. A copper wire antenna is a type of antenna that is made from copper wire, which is used to transmit radio frequency (RF) signals within FM frequency range. We are using copper wire antenna to transmit data clearly and accurately without any kind of noise, distortion, interference etc;

3.2.1 Advantages of the Copper Wired Antenna

• **High Conductivity**: when compared to other antennas copper has greater Conductivity.

• **Corrosion Resistance**: copper has better corrosion resistance compared to Aluminum, Iron etc;

Ductility and Malleability: Copper is a highly ductile metal, meaning it can be easily stretched and drawn into thin wires without breaking. It is also malleable, allowing it to be bent and shaped easily. These properties are a huge advantage in antenna construction, as it's simple to work with and can be formed into various shapes and designs, such as dipoles, loops, or yagis, to suit specific frequencies and applications.

• **Tensile Strength**: Copper has high tensile strength, so it's resistant to stretching and breaking under mechanical stress.

3.3 Microphone

In our project FBCR Microphone is used as the source to the whole system. It has an inbuilt transducer which is used to convert the input voice signal into output electrical signal.

3.4 USB Connector

3.4.0 Overview

We are using USB Connector to connect the Microphone to one of the USB Socket of Raspberry Pi. USB, i.e., universal serial bus is used since it's wide range usage and more number of USB sockets in raspberry pi.

3.4.1 Versions of USB Connector

The version is different from the Physical shape of the Connector.

1. **USB 1.x**: The first widely used version, with a maximum speed of 12 MBPS.

2. **USB 2.0**: It's the most common version, with maximum speed of 12MBPS.

3. **USB 3.x**: This version is recognized by the blue color inside the port. USB 3.0 (now called USB 3.2 Gen 1) has speed 5GBPS, while USB 3.2 Gen 2 has 10 GBPS.

4. **USB4**: It's the latest version, uses USB-C Connector. Its speed is 40GBPS.

3.4.2: Types of USB Connectors. The type of Connector is determined by its shape. The various types are used for various uses and device sizes.

1.USB-A: This is the most common USB Connector. It has a flat, rectangular shape and is typically found on the host side of a connection.

2. USB-B: It's of the square shape. This is mainly used for larger peripheral devices like printers.

3. Mini-USB: This is mainly used for older portable devices like digital players. It replaced micro-USB largely.

4.Micro-USB: It's a small, 5-pin Connector. It's a standard for many smartphones, tablets and other compact devices before existence of USB-C.

5.USB-C: It's the latest and versatile version of USB. It's reversible, means it can be connected in either direction. It rapidly replaced micro-USB and became new standard for many devices like laptops, smartphones etc; Since it supports much higher data speeds, greater power and it also can carry display signals.

3.4.3: Advantages of USB Connectors

It has Many Advantages as listed below:

1.Universal Compatibility.

2.Plug-and-Play Functionality.

3.Combined Data and Power.

4.High Data Transfer Speeds.

5.Versatility.

CHAPTER-4

SOFTWARE DESCRIPTION

4.0 Introduction

The Embedded systems runs only in the usage of software with hardware. As Our Project FBCR is belongs to embedded systems, it also runs with the integration of both hardware and software.

The implementation of our project FBCR is largely dependent on the Software combination with the Hardware. We have used two kinds of software, they are:

1.Raspberry pi imager

2. PiFM Software

This chapter deals with the types of Software that are necessary for our project and their detailed explanation about how to install them, how to run them and how they were used in our project FBCR.

4.1 Raspberry Pi Imager

4.1.0 Overview

Raspberry Pi Imager is a free software tool used to install the Operating system into the Micro SD card. It's free of cost for everyone.

4.1.1 Features and Uses of Raspberry Pi Imager

1. Easy OS Installation: The Main Function of the Raspberry Pi Imager is to download and write the Operating system (OS) images into the SD card. We can install any type of Operating system from the list like Raspberry pi OS, Linux OS etc.; This tool automatically handles the download and writing process, which saves you from manually finding and downloading image files.

2. Automatic OS Catching: Once you have downloaded the Operating system (OS) once, raspberry Pi Imager saves it locally. So that you don't have to download the again to write the same OS to multiple SD cards.

3.Custom Image Support: For advanced users the Imager also allows you to flash a custom OS image that you've downloaded yourself. You can simply select the image file from your computer and write it to the SD card.

3. Advanced Configuration: The image provides an "Edit Settings" option before you write the image, where you can Pre- Configure important settings for a headless setup (a setup without Monitor or Keyboard). You can set up your Wi-Fi credentials, enable SSH for remote access, and change the default username and password. This is a huge time-saver as it allows you to get your Raspberry Pi on the network and accessible without ever needing to connect a display.

4.1.2 Advantages of the Raspberry Pi Imager

The Raspberry Pi Imager is a very popular and widely used tool for setting up a Raspberry Pi. It offers several key advantages that make it the recommended method for both beginners and experienced users they are:

1. User-Friendly Interface: The Main Advantage is very simple. The Imager has a clean, straightforward graphical user interface (GUI). Thus it makes very simple for the beginners to install an OS into the Micro SD card without needing to use complex command-line tools.

2. Streamlined Process: Due to this process several manual steps are automated. It automatically downloads selected OS, verifies image, and writes it to the SD card.

3. Offline Catching: When an OS is downloaded, the image stores a cached copy. This is a significant advantage since as we can write same OS on Multiple SD cards without re-downloading the file each time.

4. Pre- Configuration for Headless Setup: This is a standout feature. By clicking on the "Edit Settings" or gear icon, you can set up Wi-Fi credentials, enable SSH (Secure Shell) for remote access, and change the default username and password before the image is written. This saves a lot of time to the users.

5. Official Support and Reliability: As this is an official tool from the raspberry pi foundation, the Imager is highly reliable and is regularly updated. It provides access to latest official Raspberry Pi OS version and other officially supported distributions.

4.1.3: Setting up an Raspberry Pi Imager

Broadly it consists of three main parts:

1.Choose Operating system.

2.Choose Storage

3.Write.

Step-by-Step procedure

1. Go to the official website: Open your web browser and navigate to raspberrypi.com/software.

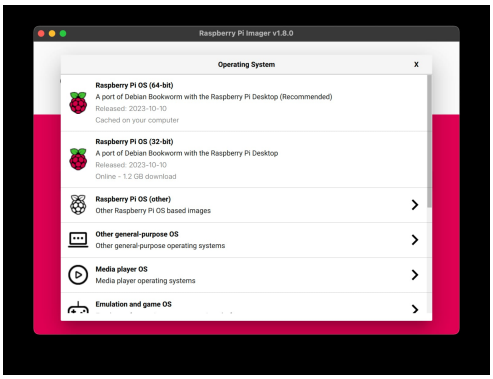
2. Select your operating system: Look for the link for Raspberry Pi Imager that matches your computer's operating system (like raspberry Pi OS, Linux OS etc;) and click it to download the installer.

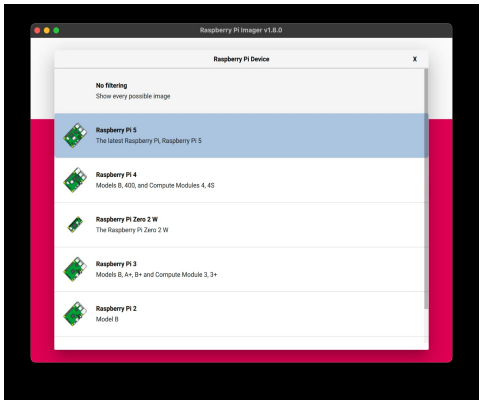
3. Run the installer: Once the download is complete, click on the downloaded file to launch the installer.

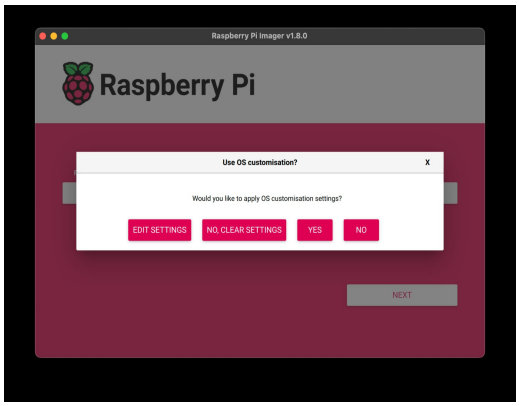
4. Follow the prompts: Follow the on-screen instructions to complete the installation process.

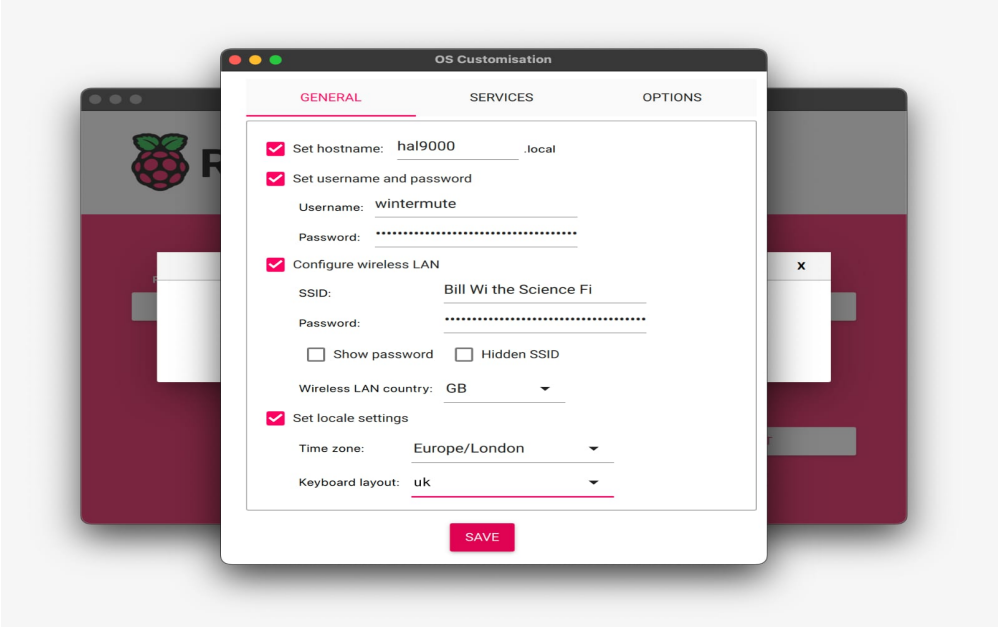
5. Launch the Imager: After installation, you can launch the Raspberry Pi Imager from your applications menu by clicking the icon or running the command rpi-imager.

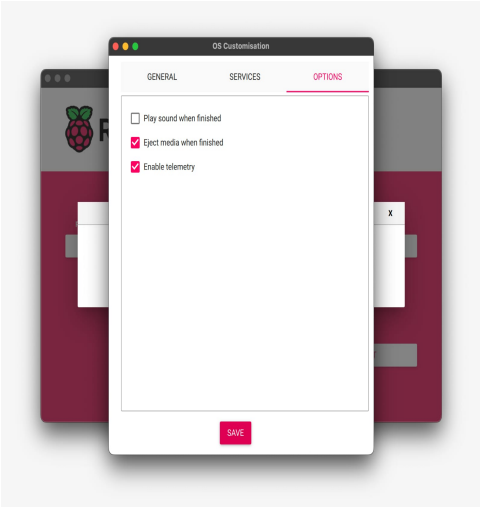
Once the Imager is installed, you can use it to flash an operating system onto an SD card for your Raspberry Pi.

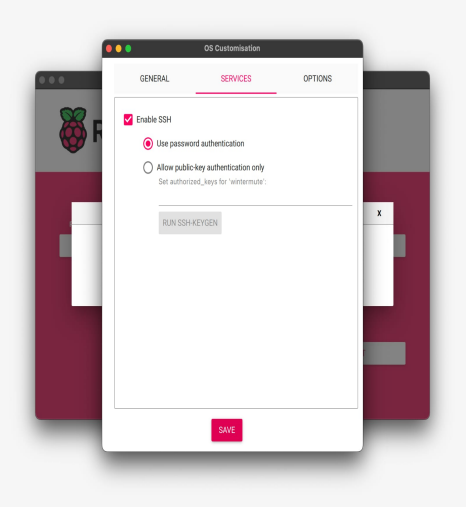


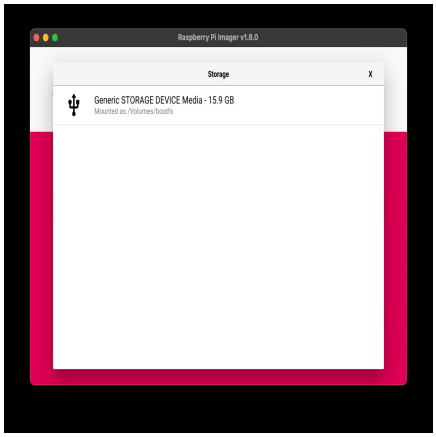


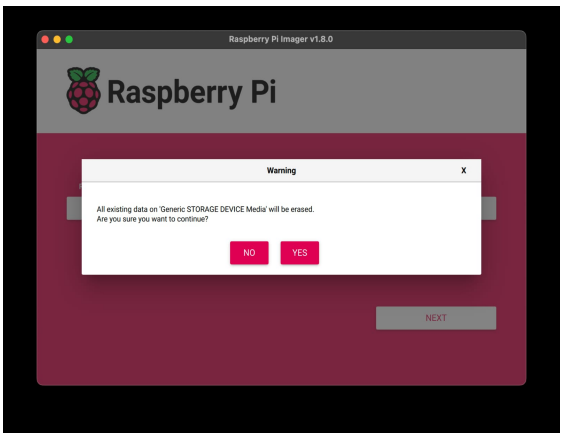



OS Customisation
GENERAL
SERVICES
OPTIONS
Set hostname:
hal9000
.local
Set username and password
Username:
wintermute
Password:
Configure wireless LAN
SSID:
Bill Wi the Science Fi
Password:
Show password
Hidden SSID
Wireless LAN country:
GB
Set locale settings
Time zone:
Europe/London
Keyboard layout:
uk
SAVE


OS Customisation
GENERAL
SERVICES
OPTIONS
Play sound when finished
Eject media when finished
Enable telemetry
SAVE


OS Customisation
GENERAL
SERVICES
OPTIONS
Enable SSH
Use password authentication
Allow public-key authentication only
RUN SSH-KEYGEN
SAVE


Raspberry Pi Imager v1.8.0
Storage
Generic STORAGE DEVICE Media - 15.9 GB


Raspberry Pi Imager v1.8.0
Raspberry Pi
Warning
All existing data on 'Generic STORAGE DEVICE Media' will be erased.
Are you sure you want to continue?
NO
YES
NEXT

4.2 PiFM Software

PiFM is a "Software-defined radio" application for the Raspberry Pi. Instead of requiring a dedicated radio transmitter chip, it leverages the Pi's built-in Hardware and Software to generate and broadcast an FM signal. The core of PiFM's functionality lies in the Raspberry Pi's PWM (Pulse Width Modulation) or GPIO (General Purpose Input/Output) pin 4. The Software Manipulates the clock signal on this pin, rapidly switching it on and off. This rapid switching creates a square wave, which, when a simple wire antenna is attached, radiates as an FM signal. The Software then modulates this signal by varying the frequency of the switching to Encode Audio Data.

Before you begin, you'll need a Raspberry Pi with a fresh installation of Raspberry Pi -OS, a power supply, and a short wire (about 20-40 cm) to use as an Antenna. Connect the wire to GPIO pin 4 (pin 7 on the header).

Here are the step-by-step instructions for installing and running the original PiFM software:

1. Update your system:
Open a terminal on your Raspberry Pi and run the following commands to ensure all your system packages are up to date.

```
sudo apt update
sudo apt upgrade -y
```

2. Install dependencies and clone the software:
PiFM relies on other programs to function. Install the required build tools and then download the PiFM source code from a repository like the one by Markondej.

```
sudo apt install make gcc g++ build-essential
git clone https://github.com/markondej/fm_transmitter
```

3. Compile the software:

Navigate to the new directory and compile the source code into an executable program. cd fm_ transmitter make:

4. Run the transmitter:

Once the compilation is complete, you can run the program. The following command broadcasts a sample file included with the software on a specific frequency.

sudo ./fm_transmitter -f 100.6 star_wars.wav

Note: Change 100.6 to a clear, unused frequency in your area. You can find one by using an FM radio to scan for static.

Steps involved in the Github :

1. Clone the Repository:Use the git clone command to download the project's entire repository to your Raspberry Pi. This command copies all the project's files and history from GitHub.

For example, to clone the markondej/fm_transmitter project, you would use:

git clone https://github.com/markondej/fm_transmitter

This will create a new folder named fm_transmitter in your current directory

2. Build the Program ⏳

Navigate into the newly created folder and use the make command to compile the C source code into a runnable executable file. This process turns the human-readable code into a machine-executable program.

cd fm_transmitter

make

If the compilation is successful, you will have a new executable file, typically named FM_transmitter or Pi_FM_RDS.

4.4. Run the Transmitter

Once the program is compiled, you can run it using Sudo because it needs direct access to the Raspberry Pi's GPIO pins and hardware clocks. The basic command structure includes the program name, the frequency, and the audio file.

sudo ./fm_transmitter -f 100.6 sound.wav

•sudo: Grants the command superuser privileges.

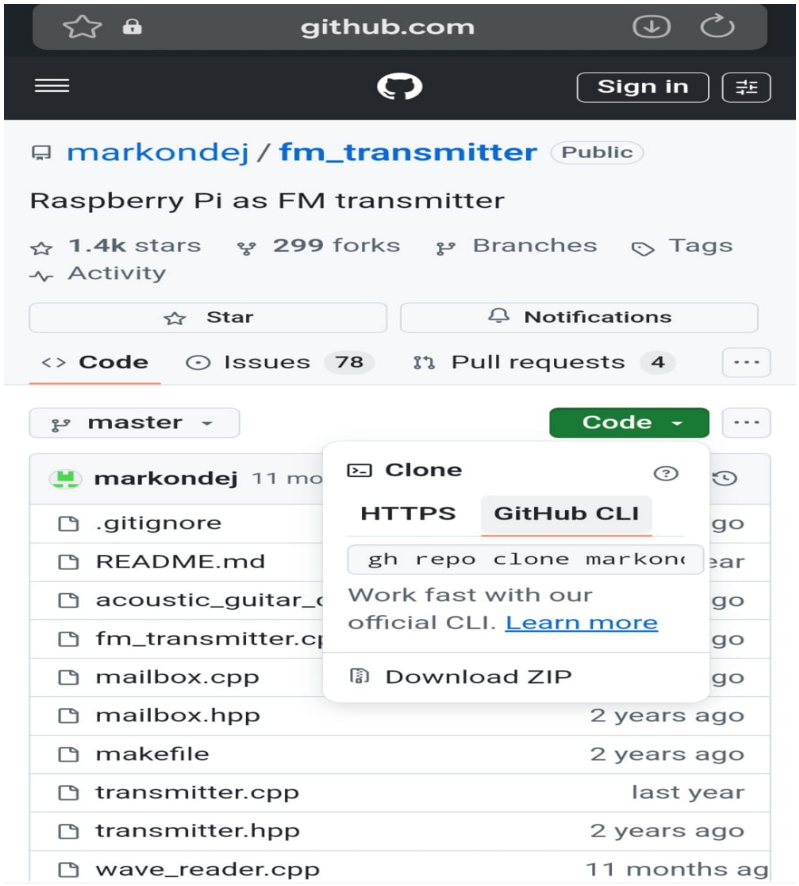
•./fm_transmitter: Runs the executable you just compiled.

•-f 100.6: Sets the transmitting frequency to 100.6 MHz.

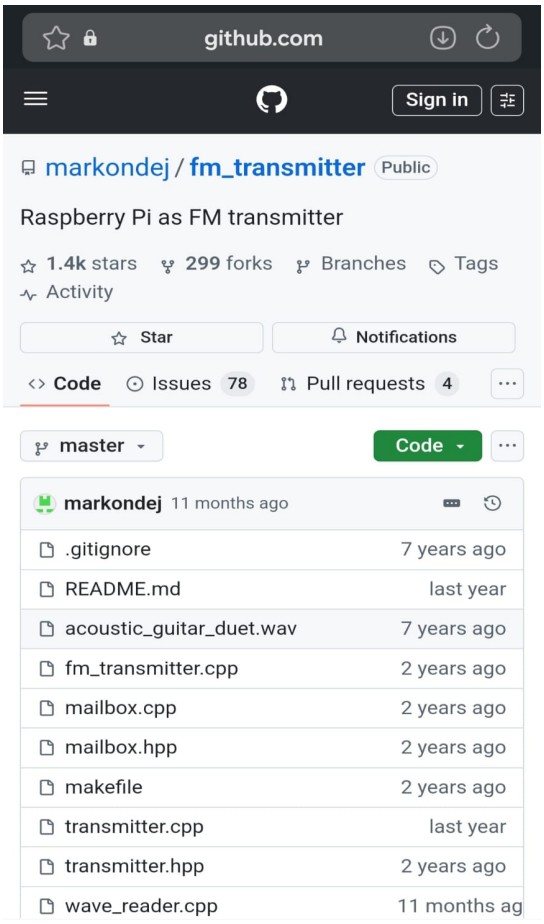
•sound.wav: Specifies the audio file to broadcast.

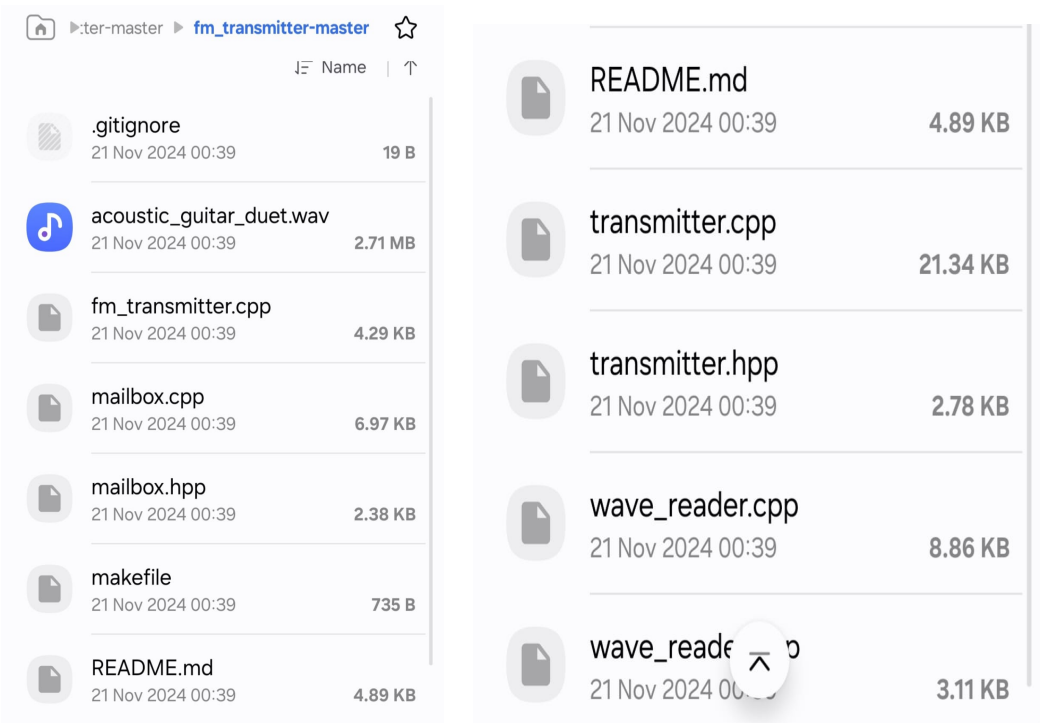
5. Connect the Antenna

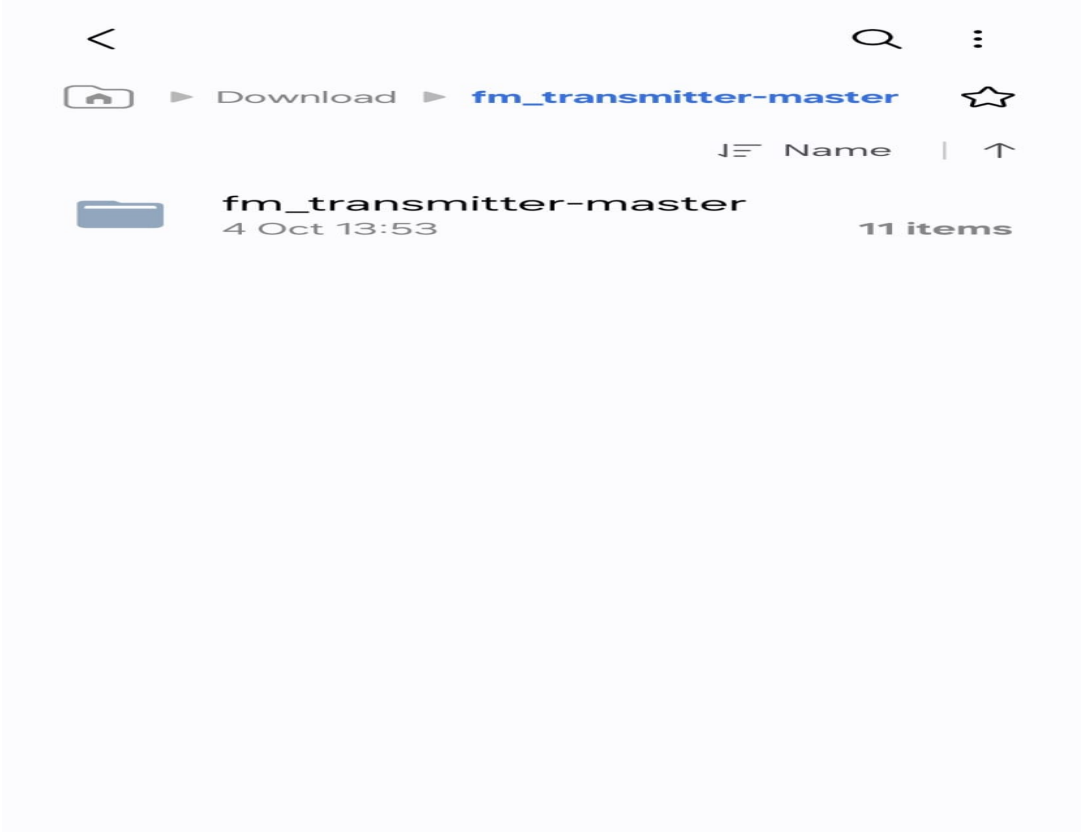
For the best range, you need to connect a simple antenna to the GPIO pin on your Raspberry Pi. A 20-40cm length of plain wire connected to GPIO Pin 4 (Pin 7 on the header) is all you need.


github.com
Sign in
markondej / fm_transmitter
Public
Raspberry Pi as FM transmitter
1.4k stars
299 forks
Branches
Tags
Activity
Star
Notifications
Code
Issues 78
Pull requests 4
master
Code
markondej 11 mo
Clone
HTTPS
GitHub CLI
gh repo clone markon
Work fast with our official CLI. Learn more
Download ZIP
.gitignore
README.md
acoustic_guitar_
fm_transmitter.cp
mailbox.cpp
mailbox.hpp 2 years ago
makefile 2 years ago
transmitter.cpp last year
transmitter.hpp 2 years ago
wave_reader.cpp 11 months ag


All
1 selected
ge ▶ Download ▶ fm_transmitter-master.zip
fm_transmitter-master
21 Nov 2024 00:39
11 items
Cancel
Extract


github.com
Sign in
markondej / fm_transmitter
Public
Raspberry Pi as FM transmitter
1.4k stars
299 forks
Branches
Tags
Activity
Star
Notifications
Code
Issues 78
Pull requests 4
master
Code
markondej 11 months ago
.gitignore 7 years ago
README.md last year
acoustic_guitar_duet.wav 7 years ago
fm_transmitter.cpp 2 years ago
mailbox.cpp 2 years ago
mailbox.hpp 2 years ago
makefile 2 years ago
transmitter.cpp last year
transmitter.hpp 2 years ago
wave_reader.cpp 11 months ag


:ter-master ▶ fm_transmitter-master
Name
.gitignore
21 Nov 2024 00:39 19 B
acoustic_guitar_duet.wav
21 Nov 2024 00:39 2.71 MB
fm_transmitter.cpp
21 Nov 2024 00:39 4.29 KB
mailbox.cpp
21 Nov 2024 00:39 6.97 KB
mailbox.hpp
21 Nov 2024 00:39 2.38 KB
makefile
21 Nov 2024 00:39 735 B
README.md
21 Nov 2024 00:39 4.89 KB
README.md
21 Nov 2024 00:39 4.89 KB
transmitter.cpp
21 Nov 2024 00:39 21.34 KB
transmitter.hpp
21 Nov 2024 00:39 2.78 KB
wave_reader.cpp
21 Nov 2024 00:39 8.86 KB
wave_reade
21 Nov 2024
3.11 KB



4.3 conclusion

By the end of this chapter, we got to know the importance of Software in the execution of our project FBCR and the installation process of both the Software.

CHAPTER-5
CIRCUIT DESIGN AND INTEGRATION

5.0 Overview

The circuit design of the FBCR system is fundamental to its operation ensuring that the various hardware components work in harmony to achieve the radio FM signal transmission. The system incorporates several interconnected components including the Raspberry Pi, antenna and microphone, all which communicate to create an reliable FM radio transmitter. Proper wiring, power supply management, and signal transmission are critical to the efficient functioning of the system.

5.1 Circuit diagram

The circuit consists of the following connections:

• Raspberry Pi to microphone: The input we as speech audio signal is converted into electrical signal by the transducer present in the microphone. Mic is connected to the USB 2.0 port of the Raspberry pi using USB Connector. USB Port is of the type class-c. The electrical signal is then passed to the raspberry pi. Now raspberry Pi modulates the signal using the FM modulation, makes signal suitable for transmission and sends it to the antenna.

Frequency Modulation: The process of changing the frequency of the high frequency carrier signal in accordance with the instantaneous amplitude of the input message signal is called "Frequency Modulation".

• Raspberry to Power supply: The Power supply is connected to raspberry pi by using an DC adapter of ratings 5v, 2.5A, and 12.5 watts which supplies the required power to the entire system. Power supply is connected to the Micro USB socket of the Raspberry pi which supplies required power to the antenna and microphone.

• Raspberry Pi to Antenna: Copper wired antenna is connected to GPIO 4 (pin 7 on the header), GPIO 4 is designed for this specific function. After an OS is downloaded into an Micro SD card and inserted into SD card socket on the back side of the Raspberry Pi, the signal sent to antenna and it transmits signal into the free space.

5.2.2 Receiver

Receiver is defined as "a circuit that accepts signals from a transmission medium (which can be wireless or wired) and decodes or translates them into a form that can drive local circuits".

In our project FBCR the receiver is same as the receiving unit in the existing FM Radio system. We can receive and listen the signal in the mobile phones, systems, etc., as we listen to normal FM. But we recommend to use speakers or headphones for listening the FM signal so that we can listen clearly.

In the FM radio app, we have to select the FM Channel set at the transmission part to listen in our phone.

5.2.3 Channel

Channel is defined as "the medium or path through which a message travels from a sender to a receiver".

It is classified as follows:

1.Guided Media: These are defined as "physical pathways, such as cables, that direct and confine data signals along a defined route, providing higher speeds, greater security, and more stable communication than unguided media".

2.Unguided Media: These are defined as "a transmission method that uses electromagnetic waves (like radio waves, microwaves, and infrared) to send signals through the air or free space, rather than relying on a physical conductor or medium like cables".

In our project FBCR we are using unguided Media as the transmission media. It's free space or air.

5.2 Circuit Explanation

Any communication system broadly consists of three main parts, namely:

1.Transmitter
2.Receiver
3.Channel

5.2.1 Transmitter

The transmitter is defined as the "a circuit that accepts signals or data in and translates them into a form that can be sent across a medium (transmitted), usually over a distance".

In our project FBCR we are designing a reliable FM radio transmitter using raspberry Pi. Input is the voice speech we speak in the Microphone. The raspberry Pi acts as a transmitter, which modulates the signal, amplifies the signal and makes the signal suitable for transmission. Then sends it to the antenna.

In the web server we select the specific FM value within the FM range of 88MHz-108MHz. Except default values any value within the range we can select as our wish.

5.3 Hardware-Software Integration

5.3.0 Overview

In our project FBCR Integration of hardware with software is essential for its functioning., Two Fundamental Software are used in our project FBCR, namely:

1.Raspberry Pi Imager
2.PiFM Software

5.3.1 Key elements in the integration

1.Device Drivers and Firmware.
2. Communication Protocols.
3. Application Programming Interfaces (APIs).
4. Middleware etc;

5.3.1 Raspberry Pi Imager and raspberry pi

Raspberry Pi doesn't have hard drive, so we need Micro SD or SD card for storage. The Python code which is essential to run the Raspberry Pi is also - can only be written into raspberry Pi only through SD card. This is done by installing an Operating system (OS) into the SD card. The installation is done by raspberry Pi Imager. The steps for setting up an Raspberry Imager is already covered in chapter-4.

We are using raspberry OS in our project FBCR since it's from the official Raspberry Pi foundation.so they regularly update the Operating system.

The Role of Operating system (OS)

The OS (Raspberry Pi OS) is the master software that runs on the Raspberry Pi. It provides the environment in which all other programs, including our C and Python scripts, can run.

• Hardware Management: The Operating system (OS)manages all low-level components like GPIO pins, USB Ports, etc.

• Software Foundation: This provides the necessary tools and libraries (like apt-get and git) required to install and compile the PiFM software.

•Multitasking: This allows multiple programs run simultaneously. This is what enables C program to run in the background while the Python web server is also active and listening for commands.

In short, the Raspberry Pi Imager prepares the "brain" (the OS) and places it into the "body". The OS then provides the foundation upon which custom our "college radio" software will operate. Without this initial integration, our project FBCR would not be able to function.

5.3.3 Integration of Pi FM software with the Hardware

The integration of Pi FM software with the hardware is the core of our project FBCR. This is the stage where the abstract code becomes a physical FM transmitter. This integration happens at a very low level, directly manipulating the Pi's hardware.

Working of Pi FM with the hardware

The integration is the direct relationship with the hardware where the software takes the command of specific hardware components to perform a function.

1. GPIO Pin Control: The c code particularly Targets the GPIO pin 4. This pin is not a regular data pin- it is connected to a hardware clock generator that can be programmed to output a specific frequency. The software controls this clock generator precisely.

2. Frequency Modulation: The software doesn't just turn the pin on and off. It constantly and rapidly changes the output frequency of the pin based on the audio data it is processing. This is the Frequency Modulation (FM) part of the process. The variations in the frequency of the signal correspond to the variations in the amplitude of the sound wave you want to transmit. The software reads the audio file's data and "modulates" the hardware signal accordingly.

3. The Antenna: The last part of the hardware is the antenna which enables the signal broadcasting. The signals generated by the GPIO are very and it's impossible to transmit beyond small distance. So the copper wired antenna acts as a radiator which amplifies and transmits the signal by converting electrical signal into electromagnetic waves that can travel through the air to FM radio receiver.

4. Resource Management: The software also manages the resources of the Raspberry Pi the Sudo command is required to run the program because it needs direct, low-level access to the hardware that a regular user doesn't have. The software also needs to manage the CPU usage and memory to ensure a stable, uninterrupted transmission.

In essence, the C code is the specialized driver that translates audio data into the hardware commands necessary to physically generate and broadcast a radio signal from the Raspberry Pi's GPIO pin. The hardware provides the capability, and the software provides the instruction, making them a single, integrated system.

5.4 Conclusion

By the end of this chapter, we got to know about the

•The circuit diagram employed in the project FBCR.

•The working of the circuit.

•The integration of software with hardware.

•Integration of Raspberry Pi OS with the Raspberry Pi.

•Integration of the Pi FM software with the hardware.

CHAPTER-6

BLOCK DIAGRAM AND DESIGN WORKFLOW

6.0 Overview

This chapter explains the fundamental block diagram of our project FBCR and overall Design workflow of our project.

The block diagram explains the complete architecture of the system. The block diagram shows briefly the connections of the components.

The Design workflow explains about the complete process from the starting source to the FM radio receiver. The design workflow is pictorially depicted by means of the flowcharts.

6.1 The Importance of Block Diagram

The Block diagram is a high-level visual representation that shows the relationship between different parts of a system. For our FBCR project, it is essential because it provides a clear, conceptual overview of how all the components—both hardware and software—work together.

1. Conceptual Clarity: A Block diagram simplifies a complex system into a series of interconnected blocks. Each block represents a component (like the Raspberry Pi, Antenna, or Web Server), and the lines connecting them represent the flow of information or signals.

2. Communication and Collaboration: In a team project, a block diagram is a powerful tool for communication. It allows all team members, regardless of their specialization (hardware, software, or documentation), to quickly grasp how their individual work fits into the overall system. It helps to prevent misunderstandings and ensures everyone is on the same page.

3. Troubleshooting and Debugging: When something goes wrong, the block diagram is a first-line diagnostic tool. If the signal isn't transmitting correctly, you can follow the path on the diagram:

•Is the Python server sending the correct command?

•Is the C program receiving the command and executing it?

•Is the C program correctly manipulating the GPIO pin?

•Is the antenna properly connected to the pin?

By visually tracing the problem, you can quickly identify the faulty component or integration point.

4. Planning and Design: Block diagram helps you plan the project effectively. It forces you to define all the components and their interactions, which helps you identify potential problems or missing parts early in the design phase. It serves as a blueprint for your project.

6.2 The Fundamental Block Diagram of the system

The Block diagram of our project FBCR is shown below and the block diagram consists of the following blocks:

1.Microphone.

2.Raspberry Pi

3.Regulates Power supply (RPS).

4.Antenna

Functioning of each Block

1. Microphone: The source to the whole system is given through the Microphone. The source is the audio speech we speak in the Microphone. Microphone has in-built transducer of the type electrical transducer. The transducer converts physical quantity (audio signal) into the electrical signal. The Microphone is connected to the USB

socket of the Raspberry Pi of the type Class-C by using the USB connector. Microphone gets the required power supply from the Raspberry Pi.

2. Raspberry Pi: The Raspberry Pi is the heart and soul of the whole system. All the major functions are done by the raspberry pi itself. The raspberry Pi does all the functions with integration of the two software's, Raspberry Pi Imager and Pi FM software. Raspberry Pi Imager is used to install the operating system (OS) into the SD card. SD card is placed into the Micro SD socket present on the back side of the raspberry Pi. In our project FBCR we are using the Raspberry Pi OS. The operating system is used to write the python code into the raspberry pi. The PIFM software is used to control the web server using the separate python code and an c code. The GitHub website gives the required c code and python code to function the web server, so that we can control the transmitter from any device by typing the IP address of the raspberry pi.

2. Regulated power supply (RPS): The power required by the entire system is supplied by the RPS. We are using a single power supply for the entire system since the power required by the blocks antenna and microphone is supplied through the raspberry Pi.

3. The power ratings of the raspberry pi are: power supply of 12.5 watts, 5V and 2.5A. The power supply is connected to raspberry Pi through the Micro USB socket of the raspberry Pi.

4. Antenna: The antenna is connected to the GPIO pin no.4 of the raspberry pi. The type of antenna we are using is the copper wired antenna of the length 2-3 meters of length. The antenna transmits FM Radio signal into the free-space or air. It receives the signal from the raspberry pi.

This is the brief description about the block diagram of our project FBCR, the functioning of each block, and the interactions between the blocks.

6.3 Design workflow

The Importance of defining a workflow for our FM Based College Radio (FBCR) project lies in its ability to transform a conceptual idea into a reliable, functional system through structured, repeatable steps.

A well-defined workflow provides the following crucial benefits

1. Ensures Reproducibility and Reliability: A project is only successful if it can be set up and operated consistently. The workflow documents the exact sequence of actions, ensuring that anyone following it—from the initial setup to daily operation—achieves the same result.

2. Streamlines Team Collaboration: In a team setting, a workflow clarifies who is responsible for each part of the project and when they need to act, minimizing bottlenecks and confusion.

•Clear Hand-offs: It defines the input and output for each stage. For example, the software team knows their input is the fully compiled Pi FM C binary, and their output is the tested Python web interface. This prevents the hardware team from waiting indefinitely for software components.

•Parallel Development: The team can work on the C code compilation (low-level) and the Python web server (high-level interface) simultaneously, knowing how they will integrate later.

3.Simplifies Troubleshooting and Maintenance: A logical workflow is the best defense against complex errors. When a problem occurs, the workflow serves as a checklist for diagnosing the issue.

•Isolation of Failures: By separating the system into sequential stages (e.g., Hardware Setup, C Code Functionality, Python Control Layer), you can isolate which part is failing. If the C code transmits successfully from the command line but not from the web server, the failure point is narrowed down to the Python-C integration step.

3. Provides Project Documentation and Scaling: The documented workflow becomes a core piece of project documentation that can be handed off for future maintenance or expansion (e.g., adding a playlist feature or RDS).

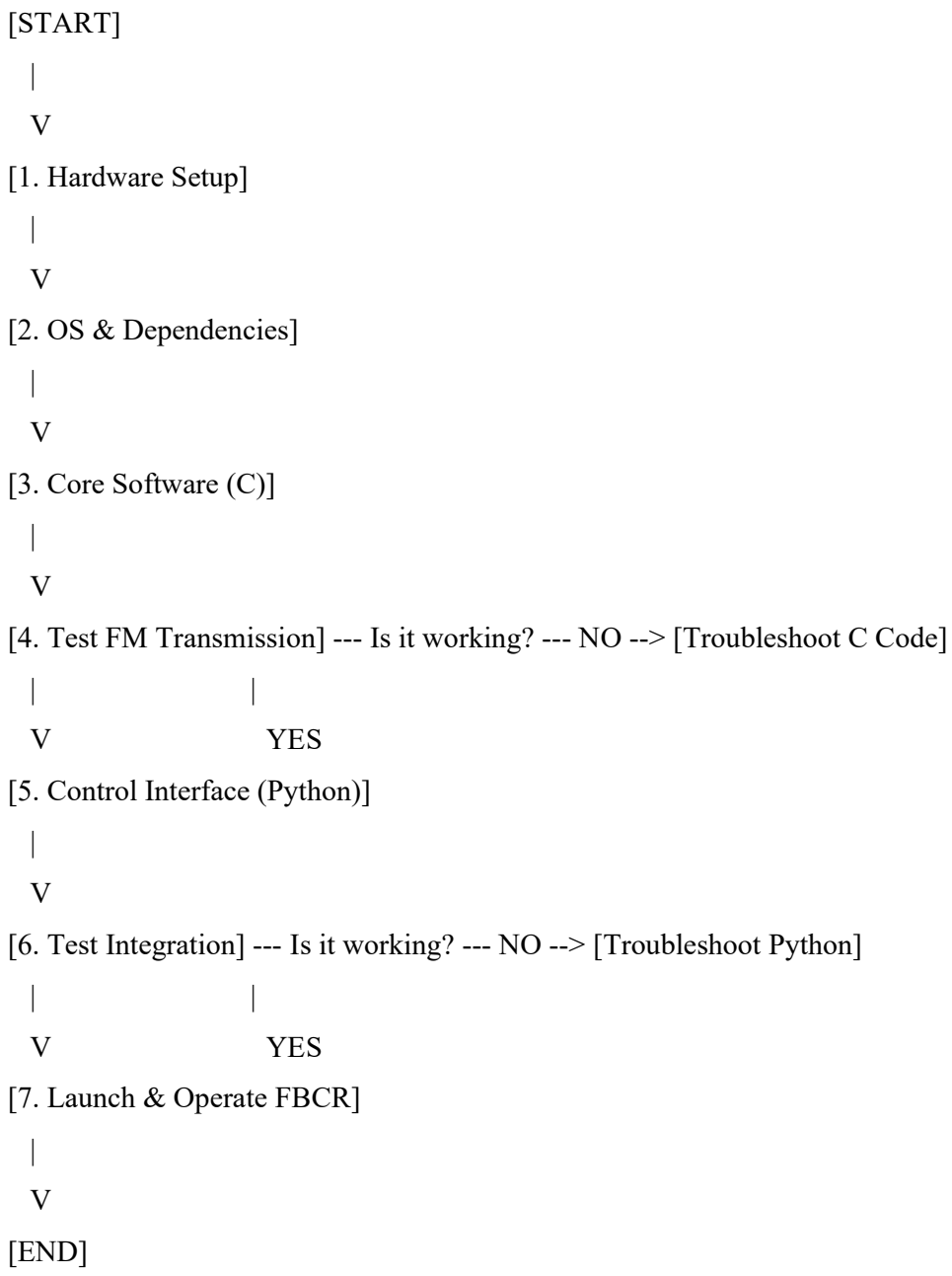
•Training: New team members can quickly learn how the system operates by following the established procedures.

•Scaling: If you decide to move to a more advanced transmitter or a different Raspberry Pi model, the workflow provides the existing framework to follow, making the transition easier.

The design workflow for our FM Based College Radio (FBCR) project is a clear, step-by-step process that ensures all components are built, tested, and integrated correctly. A flowchart provides a visual map of this journey.

Here is a simplified flowchart of the FBCR design workflow:

FLOW CHART



1. Hardware Setup

This is the physical foundation of our project. The first step is to prepare the Raspberry Pi by setting up the operating system (OS) on the Micro SD card using Raspberry Pi Imager. Then we connected the antenna to the GPIO pin 4 and set up the power supply. At this stage, we have all the physical parts ready to receive software instructions.

2. OS & Dependencies

With the hardware ready, the next step is to configure the software environment. This involves updating the OS and installing essential tools like Git (to download code from GitHub) and build-essential (to compile the C code). This prepares the system to be a development and runtime environment for our project.

3. Core Software (C)

This is where the magic begins. You use the git clone command to download the Pi FM C code from its repository. After that, compile the source code into a runnable program using the make command. This compiled program is the "engine" that will directly control the Raspberry Pi's hardware.

4. Test FM Transmission

Before you build the web interface, it's crucial to test the C code on its own. You should run a simple command-line test using Sudo. /Pi_fm_rds with a test audio file. If the transmission is successful and you can receive the signal on an FM radio, you know the core functionality works. If not, you need to troubleshoot the C code or the hardware connection.

5. Control Interface (Python)

Once the core C program is verified, you move on to building the Python web server. This involves installing the Flask library and writing the Python script to create the web interface. The script will be designed to listen for user input from a web page (e.g., a frequency or an audio file to play).

6. Test Integration

This is the final test of your complete system. You run the Python web server and use a web browser on another device to send commands to it. The key test here is whether the Python script correctly executes the C code using subprocess. run (). If you can successfully change the frequency or play a song from the web page, the integration is a success. If not, you need to troubleshoot the Python code to ensure it is correctly formatted and sending the right commands.

7. Launch & Operate FBCR

Once all components are working together, your college radio project is ready to go live! You can now operate the entire system remotely and begin your broadcasts.

6.4 Conclusion

By end of this chapter, we got to know the importance of the block diagram in our project FBCR, the blocks present in the block, the design workflow of the system, flowchart representing the design workflow, it's importance.

CHAPTER-7

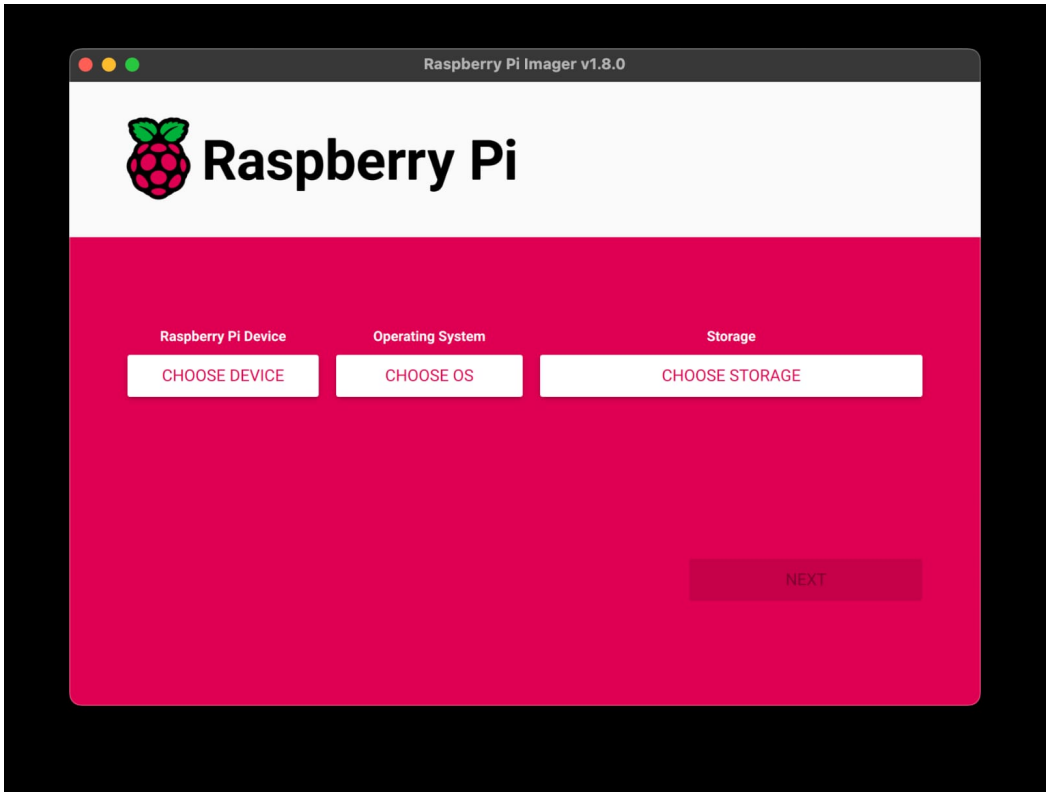
THE WORKING METHODOLOGY

7.0 Overview

This chapter deals with the complete working of the system FBCR in a step-by-step procedure:

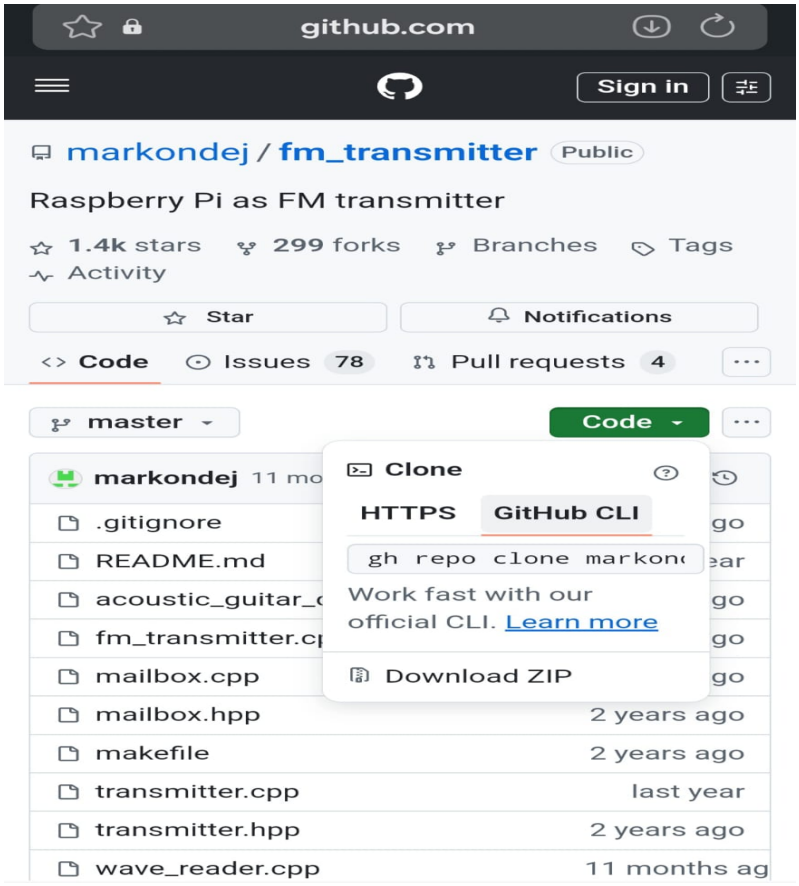
Step-1:

Install the Operating system into the Micro SD card as explained in the chapter number 4.



Step-2:

Dump the Python and c++ code into the Raspberry Pi as discussed in the chapter number 4.



Step-3:

Install the PiFM software into the Raspberry Pi as explained in chapter number 4.

Step-4:

Connect the copper wire to GPIO pin 4 of the Raspberry Pi.

Step-5:

Connect the microphone to the USB Port of the Raspberry Pi using the USB Connector.

Step-6:

Connect the power supply to the power socket of the Raspberry Pi.

Step-7:

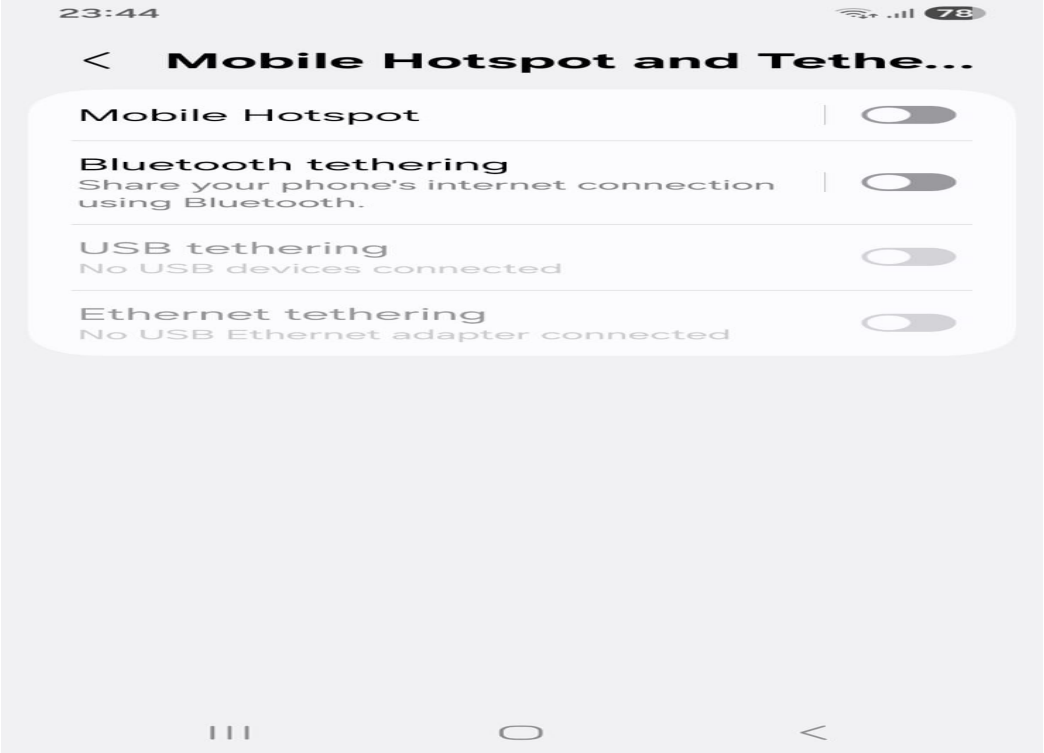
Switch on the mobile Hotspot on your Mobile phone.

Step-8:

Switch on the power supply.

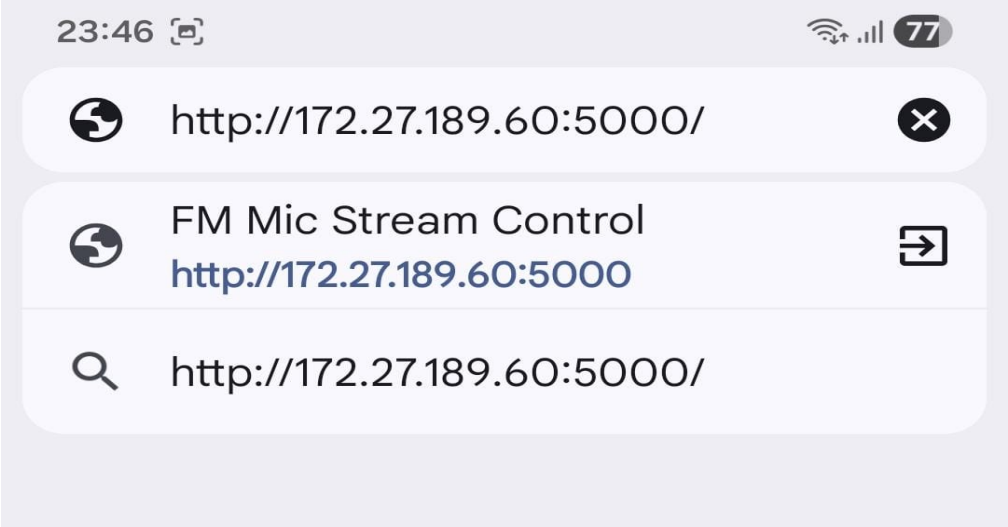
Step-9:

Connect mobile Hotspot to raspberry pi and note down the IP (internet protocol) address of the Raspberry Pi (as indicated in the info).



Step-10:

Type the IP address in the web browser (suggested one is the Google chrome).



Step-11:

A panel will be shown. Select the FM channel you want to select (like 94.7 MHz, etc.,.) from the FM range 88-108 MHz for the FM based college radio system.

Step-12:

Click on the "start streaming" icon. You can listen from any FM radio receiver like mobile phone, laptop, car FM receiver's by selecting the specific FM Channel (Ex:94.7 MHz).

7.1 Conclusion

By the end of this Chapter we learned the steps involved in the working of the project FBCR in a detailed manner.

CHAPTER- 08

FBCR- AT A GLANCE

1.Hardware Requirements

o Raspberry Pi (any model with GPIO pins).

o Audio source (USB microphone or audio file)

o Antenna wire connected to GPIO pin (for FM transmission).

o Power supply, SD card, speaker/headphones.

2. Software Setup

o Install Raspbian OS on Raspberry Pi.

o Use PiFM software to convert audio files into FM signals.

o Set a transmission frequency (usually between 88.0 MHz – 108.0 MHz).

o Use the Sudo command to execute transmission scripts (e.g., Sudo. /Pifm sound.wav 100.0).

3. Transmission Process

o Audio is either recorded live or taken from a pre-loaded file.

o The software encodes the audio into FM modulated signals.

o The GPIO pin sends this signal through a simple copper wire acting as an antenna.

CHAPTER-9

APPLICATIONS AND ADVANTAGES

APPLICATIONS

* Low-power personal FM radio station (educational/demo purposes).
* Campus or community FM broadcasting.
* Emergency alert systems in remote areas.
* Wireless transmission of audio in institutions (schools, exhibitions).
•Campus Announcements and
Notifications.
•Student-Run Radio Station.
* Emergency Broadcast System.
* Educational Tool.

ADVANTAGES

* Low-cost and easy to implement
* Fully programmable and flexible
* No need for complex analog components
* Educational value for learning RF and embedded systems
* Can be used both for transmission and reception
* Portable and compact
* Inclusion.
* Building Confidence.
* Develop Speaking & Listening Skills.
* Improving Literacy.
•Team work.

CHAPTER-10

RESULT, FUTURE SCOPE AND CONCLUSION

9.0 Result of the Project- FBCR

9.1 Future Scope

* Integration with web streaming (broadcast FM and live stream over internet).
* Voice-controlled or app-based FM channel switching.
* Extended range using amplifiers and RF filters.
* FM signal encryption and private network broadcasting.
* Combining with IoT systems for smart FM broadcasting (e.g., automatic alerts, announcements).
* Using AI to auto-schedule content and manage stations.

9.2 Conclusion

The FM-based radio using Raspberry Pi demonstrates a modern, digital approach to audio broadcasting. It replaces traditional analog systems with a software-defined, flexible model, making it an ideal tool for education, prototyping, and small-scale use. The project proves how versatile the Raspberry Pi can be in wireless communication applications with minimal hardware and cost.

REFERENCES

Websites

1. Raspberry Pi Official Website – https://www.raspberrypi.org
2. PiFM GitHub Repository – https://github.com/rm-hull/pifm
3. FM Broadcasting Regulations – https://www.trai.gov.in (India)
4. Wikipedia- https://www.wikipedia.org/
5. Tutorials point- https://www.tutorialspoint.com/

AI Tools

1.Gemini by Google DeepMind
2.Chatgpt by Open AI
3.Perplexity by Perplexity AI

APPENDIX

```
from flask import Flask, render_template_string, request, redirect, jsonify
import subprocess
import os
import signal

app = Flask(__name__)
process = None
current_freq = "100.0"

HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>FM Mic Stream Control</title>
  <style>
    body {font-family: Arial, sans-serif; background: #f4f4f4; text-align: center; padding: 50px;}
    h1 {color: #333;}
    . status {margin: 20px; font-size: 1.2em;}
    button, input[type=text] {
      padding: 10px 20px;
      font-size: 16px;
      margin: 10px;
      border-radius: 10px;
      border: none;
      background-color: #007BFF;
      color: white;
      cursor: pointer;
    }
```

```
        button. Stop {background-color: #dc3545;}
        input[type=text] {width: 100px;}
    </style>
    <script>
        function refresh Status () {
            fetch('/statuses)
            . then (response => response. Json ())
            . then (data => {
                document. getElementById('status').inner HTML = "Status: <b>" + data.
Status + "</b>";
            });
        }
        set Interval (refresh Status, 2000);
        window. Onload = refresh Status;
    </script>
</head>
<body>
    <h1> 🎙 Raspberry Pi FM Mic Stream</h1>
    <div class="status" id="status">Loading status...</div>
    <form action="/start" method="post">
        <input type="text" name="freq" placeholder="Freq" value="{{freq}} “required>
        <button type="submit">Start Streaming</button>
    </form>
    <form action="/stop" method="post">
        <button class="stop" type="submit">Stop Streaming</button>
    </form>
</body>
</html>
"""
@app. route ("/")
```

```
def index ():
    global process, current_freq
    running = process is not None and process. Poll () is None
    return render_template_string (HTML, running=running, freq=current_freq)

@app. route ("/start", methods=["POST"])
def start ():
    global process, current_freq
    stop () # Stop any existing stream first
    freq = request.form.get ("freq", "100.0")
    current_freq = freq

cmd = farecard -f S16_LE -r 44100 -D plughw:1,0 -c 2 - | Sudo nice -n -20. /pi_fm_rds -freq {freq} -PS 'LIVE' -audio -"
    process = subprocess. Open (cmd, shell=True, preexec_fn=os. setsid)
    return redirect ("/")

@app. route ("/stop", methods=["POST"])
def stop ():
    global process
    if process and process. Poll () is None:
        os. killpg (os. getpgid(process.pid), signal. SIGTERM)
    process = None
    return redirect ("/")

@app. route("/status")
def status ():
    global process, current_freq
    running = process is not None and process. poll () is None
    return jsonify ({"status": streaming on {current_freq} MHz" if running else "Stopped"})

if __name__ == "__main__":
    app.run (host="0.0.0.0", port=5000)
```